

Master in Sound and Music Computing
Universitat Pompeu Fabra

Virtualization of the Behringer Kobol Synthesizer VCO

Valentín Malpica Gómez

Supervisor: Frederic Font Corbera

August 2024



Master in Sound and Music Computing
Universitat Pompeu Fabra

Virtualization of the Behringer Kobol Synthesizer VCO

Valentín Malpica Gómez

Supervisor: Frederic Font Corbera

August 2024



Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.3	Analog synthesizer.	3
1.4	The RSF Kobol	4
1.5	The Behinger Kobol	6
2	State of the art	9
2.1	Kobol Clones Synths	9
2.1.1	Semimodular synthesizer	9
2.1.2	Modular Systems: Just VCO.	10
2.1.3	Virtual Instruments	11
2.2	Previous related works	11
3	Methodology	15
3.1	Audio plugin implementation	15
3.1.1	JUCE framework	15
3.1.2	Plugin architecture	17
3.1.3	Synthesizer architecture	20
3.1.4	Oscillator adjustment	21
3.1.5	Plugin GUI	23
3.2	Signal capture	25

4	Results	28
4.1	Time Domain Evaluation	28
4.2	Frequency Domain Evaluation	29
4.3	Subjetive evaluation	32
4.4	Plugin Evaluation	33
4.4.1	Plugin Validation	33
4.4.2	Performance testing:Stress Test	34
5	Conclusions	36
5.1	Discussion	36
5.2	Future work	37
	List of Figures	39
	List of Tables	40
	Bibliography	41
A	Schematics	43
B	Kobol_osc_js Code	45
B.1	Comparator.js	45
B.2	Rectifier.js	46
B.2.1	Main.js	48
C	Audio Plugin Code	54
C.1	Oscillator.cpp	54
D	Evaluation results	59
D.1	Kobol and KobolVCO Waveforms	59
D.1.1	A2 Waveforms	59
D.1.2	A3 Waveforms	60

D.1.3	A4 Waveforms	60
D.2	B.Kobol and KobolVCO MSError	61
E	Plugin Evaluation Log	62
E.1	Plugin Test Evaluation	62

Dedication

Special dedication to my family, to my soulmate brother Fran, and to my parents Dulce and Valentín, as without their immense and unconditional support, this would not have been possible. I would also like to thank my supervisor, Frederic Font, for his patience and assistance, as well as Xavier Serra for his support when required.

Abstract

This thesis focuses on the digital modeling of the Behringer Kobol Expander VCO, a digital analog replica of the RSF Kobol Expander I, developed by the brothers Ruben and Serge Fernandez in 1979. Due to its innovative features at the time, which allowed a continuous morphing between different waveforms, and its characteristic VCO sound, it has been chosen as the subject of this thesis. This project was developed using C++, with the help of the well-known JUCE framework. The results of an objective and subjective evaluation show that the comparison between the Behringer Kobol Expander and the digital instrument has been positive. The original essence of the continuous oscillator has been preserved. And although the behavior is not identical to the original, it can be said that it behaves in the same way as the original Behringer Kobol.

Keywords: RSF Kobol; Plugin development; VCO virtualization.

Chapter 1

Introduction

Throughout this chapter we will take a closer look at the world of analogue synthesisers. First of all, we will start with the motivations that have led me to carry out this project, as well as the objectives that we intend to achieve: the creation of a virtual instrument with the implementation of the Behringer Kobol oscillator, its comparison through an objective evaluation with the sound of the Behringer Kobol; for which C++ within the JUCE framework environment has been used.

This is followed by a description of the contextual framework of the project. We will talk briefly about synthesizers in general, starting with the RSF Kobol Expander, which was a milestone at the time, capable of continuously switching from one waveform to another (wave morphing). Its history and main technical features will be discussed, of which we will focus on the VCO. After that, we will talk about its recent replica developed by Behringer (Behringer Kobol Expander). This allows the approach to the operation of the well-known synthesizer, contributing to the preservation of this type of old analogue synthesizers.

1.1 Motivation

Nowadays, the use of analogue audio equipment is still very popular in the field of music production. Despite some drawbacks such as the fragility of analogue systems,

their high price and few copies, and the impossibility to save configurations/presets, analogue audio processing allows the approach to a unique non-linear sound that introduces into the processed signal the sounds that many engineers and producers are looking for nowadays. Thanks to the virtual analogue modelling techniques, it has become possible to replicate the sound of analogue equipment in a digital environment. Regarding this, the Behringer brand has recently launched a series of low-cost analogue synthesiser ‘clones’, allowing certain characteristics of these instruments to be digitised. This is precisely where this research is situated. The Behringer Kobol expander allows us to approach the characteristics of the original RSF Kobol. In particular, we are concerned here with the continuous change from one waveform to another and the original sound of the VCO. At its launch RSF Kobol was a milestone for wave synthesizers due to its unique wave morphing of the oscillator.

1.2 Objectives

Accordingly, the following objectives have been set for this thesis:

Development of a virtual instrument recreating the Behringer Kobol Oscillator: The primary objective of this research is to design and implement a virtual instrument that accurately replicates the oscillator of the Behringer Kobol synthesizer. This involves not only recreating the oscillator’s unique sound characteristics but also ensuring that the digital model faithfully emulates the analog behavior of the original hardware. The result will be a virtual instrument capable of producing sounds indistinguishable from those generated by the physical synthesizer. The development of the digital modeling of the Voltage-Controlled Oscillator (VCO) requires an understanding of both analog synthesis techniques and digital signal processing. This modelling will serve as the foundation for the virtual instrument, ensuring that the digital recreation is as authentic as possible.

Objective comparison between the virtual instrument and the original Behringer Kobol: An important component of this thesis is the evaluation of the implemented

virtual instrument through a comparison with the original Behringer Kobol synthesizer. This comparison will be conducted using objective (and also a subjective) evaluation metrics and, focusing on the similarity of the synthetic sound produced by the virtual instrument in relation to the sound generated by the original hardware. The goal is to quantify the fidelity of the virtual model and identify any differences that may exist.

Learning and working with audio programming tools: An important goal of this project is to become competent in audio programming by using industry-standard tools, particularly the JUCE framework. This includes the practical implementation of the virtual instrument and learning advanced techniques necessary for developing audio software.

A final objective is to expand the state of the art and the available information related to the RSF Kobol Expander through work with the B. Kobol, as the information and bibliography on this synthesizer are practically nonexistent

1.3 Analog synthesizer.

In the 70's the sound of analogue synthesizers was everywhere, revolutionizing the world of music mechanics/industry. Many brands of manufacturers joined the analogue synthesizer business: Moog, ARP, Oberheim, Roland, Prophet, Kobol, among others.

Analogue synthesizers are electronic devices that allow you to create and modify sounds, ranging from traditional analogue, analogue/digital, purely digital, to software emulation. In an analogue synthesiser, the circuitry responsible for adjusting how the voltage-controlled oscillator (VCO) responds to control signals is based on a converter that transforms voltage into current exponentially. For the sake of clarification, a voltage-controlled oscillator is defined as an oscillator circuit whose frequency can be adjusted or varied by applying a DC input voltage [1]. This type of converter is similar to those that used to be used in electronic analogue computers, but audio oriented in this particular case.

In this regard, R. Wilson in *Make: Analog Synthesizers* talks about the fact that :

« [...]A good definition of the word analog would be “of, relating to, or being a mechanism in which data is represented by continuously variable physical quantities.” The key point we’re interested in as far as describing an analog synthesizer is “continuously variable.” An analog synthesizer creates sound by means of electronic circuit elements: resistors, capacitors, transistors, and integrated circuits»[. . .]». [2]

The advent of the digital era and the possibility of creating virtual instruments has relegated analogue instruments to the background: costly, delicate in their operation, and with their gadgets. The possibility of emulating the sound of mythical synthesizers, either through pluggins or current analogue/digital recreations of them, allows access to these synthesizers and guarantees their preservation, as well as recreating the integrated circuits of the originals. In this respect, the Behringer brand has launched affordable replicas such as the Behringer Kobol Expander (RSF Jobol Expander 1), SM-2 (Obberheim SEM-2 vocie), Behringer Toro (Moog Taurus), 2600 (ARP 2600), etc.

1.4 The RSF Kobol

In the 1970s, the French brand RSF, based in Toulouse, launched the Kobol Expander, the French ‘minimoog’. RSF was run by brothers Ruben and Serge Fernandez. They met Klaus Schulze who showed them the EMS synthesizer and would join them in the production and design of RSF synthesizers. In 1978 the first version of the Kobol appeared, aimed at a high-end market, with an aesthetic resemblance strongly influenced by the Minimoog. But with unique features for the time, such as continuous sweeping from one waveform to another: wave morphing. Many musicians used it, such as Jean-Michel Jarre, Vince Clarke, Vangelis and Peter Gabriel. However, sales were not good, because the French market was small and dominated by the big American and Japanese brands.

With the new modular system, sales increased. The rack version of the Kobol appeared in 1979, along with the Kobol Expander, initially available in four different

racks: Expander 1, which included VCO, VCF, VCA and LFO; Expander 2, which featured auxiliary modules not present in the original, such as ring modulation, sample-and-hold and additional VCA and LFO; the Programmer rack, which housed memory circuitry; and the KM8, an eight-voice mixer [3]



Figure 1: RSF Kobol Expander I front pannel

Among the main features of the RSF Kobol Expander 1 is its unique VCO. The RSF incorporates two VCOs that operate within a frequency range of 10Hz to 10kHz, which can be extended by voltage control. These VCOs include voltage inputs for external manipulation of both VCO pitch and level, along with individual outputs for each oscillator. What sets these VCOs apart is their waveform selection system; unlike the typical switched selectors found on most synthesizers, the RSF allows for continuous variation between waveforms. Available waveforms are right-to-left, triangle, rising sawtooth, square and pulse. By adjusting the control to a position between any two of these waveforms, you can achieve interesting mixes and create many unique sounds. This control can also be voltage modulated, allowing automated waveform sweeping by connecting the output of an envelope generator or low frequency oscillator to the input of the waveform control. Both VCOs are identical and capable of generating a wide spectrum of sounds, even without the use of filters. The level of each oscillator is managed by a voltage controlled amplifier (VCA), allowing independent voltage control over the amplitude of each VCO. In addition, there is a hard-sync switching function. The waveform creation possibilities are very extensive, even superior to those of current analogue synthesizers.

The Voltage Controlled Filter (VCF) employs a 24dB per octave voltage-controlled low-pass design and includes four main controls: Cutoff Frequency, Resonance, ADS

2 Amount and Keyboard Amount. It supports external voltage modulation for each control and offers an audio input for processing external sounds, making it flexible and easy to use.

The operation of the envelope generators is similar to that of Moog generators, where the final release time is linked to the initial decay control and can be turned on or off. The control range for attack and decay/release times is wide, spanning from 1ms to over 20 seconds. Another distinctive feature of the RSF Kobol Expander is its LFO which offers triangle and square wave outputs to create vibrato, trills, sweeps and similar effects. However, this range may seem somewhat restrictive, and it would be more versatile to include a wider selection of waveforms, ideally with a variable control like those on VCOs. The modulation rate is quite wide, from 0.01Hz to 100Hz, and this rate can be adjusted by voltage control. Finally, the noise generator provides white or pink noise, selectable via an internal switch[4]

1.5 The Behringer Kobol

In recent times, Behringer has developed the Kobol Expander, a monophonic analog synthesizer that takes inspiration from the 1978 RSF Kobol Expander I. This synthesizer, housed in a desktop case compatible with Eurorack, maintains the semi-modular design of its predecessor, featuring two voltage-controlled oscillators (VCOs) and a filter modeled after the iconic SSM2040 chip.

The Kobol Expander provides distinct sound-shaping options, such as waveform blending and broad modulation capabilities, making it a flexible addition to any modular setup. Despite having limited MIDI functions and no sound storage memory, the synthesizer excels in delivering warm, analog tones at an attractive price of 189 Euros. Its analog envelopes and low-pass filter offer a sound quality that echoes vintage analog synthesizers[5].

The design includes two identical VCOs known for their stability and precision, offering seven waveforms that can be continuously altered via voltage control and a front panel knob. Instead of the original UA726, a dual matched transistor along

with Tempco has been used.[6]. The wave generators are similar to the original but use TL074 Op Amps and V13700 OTAs. The pitch accuracy is maintained within 1 cent across a five-octave range. The VCO output levels are slightly lower than those typically found in Eurorack modules, as shown in Table 1:

Triangle	Shark	Sawtooth 3	Sawsqr	Square	Pulse	Pulse Varying
+1.6V	+2.5V	+3.0V	+1.0V	+2.1V	+2.1V	+2.1V

Table 1: RSF Kobol VCO levels

The broad frequency control of the VCO requires careful tuning, which can be done either by ear or with a tuner. Additionally, VCO2 includes a fine-tuning feature to synchronize the beat frequencies between both VCOs.

The Kobol's LFO is basic compared to its VCO, offering only triangle and square waveforms without crossfading. It oscillates from 0.01 to 100 Hz and can be modulated via a patch socket. While it can theoretically be used as a third oscillator, the tracking was not accurate in testing. The LFO directly modulates the VCO pitch, with control over modulation strength and options for further modulation of filter frequency and envelope parameters, allowing for some experimentation. The Behringer filter is designed using a discrete transistor replica of the SSM2040 24dB low-pass filter chip.



Figure 2: Behringer Kobol Expander front pannel

There is also a noise generator that can produce white noise. Each VCO has its own independent volume control. There are two identical ADS envelope generators. The original used SSM2050 chips with release matching decay. The Behringer version

uses a 555 timer chip and offers attack and decay/release times from 1ms to 15 seconds, with sustain ranging from 0 to +10V.

The VCA is a simple voltage-controlled amplifier that takes its audio input from the VCF and is controlled only by the second ADS envelope, with no external CV inputs or patching options. It has a final level control before the output. Behringer uses a dual OTA (V13700) for this VCA[5].

Chapter 2

State of the art

2.1 Kobol Clones Synths

Several brands have tried to emulate the mythical and original RSF Kobolo Expander I through clones. There is little information about them online. Some of these synthesizers can't even be found nowadays as they are discontinued. Here is a review of the main clones that have been developed.

2.1.1 Semimodular synthesizer

Mos-Lab Kobol Expander III.

Developed by Mos-Lab company specialised in building modular and Moog modules since 2007, based on original designs and schematics from the 60s and 70s. The shape of the synthesizer is a Moon Modular rack, with a standard power supply. [7]

Black Corporation KIJIMI:

This is a clone of the Polykobol II. The Polykobol II was a polyphonic keyboard introduced in 1982 which the Fernandez brothers were working on for years, among other reasons due to the difficulty of adapting the Kobol's electronic boards. It included new features such as a digital polysequencer, a dynamic keyboard, a K7 tape interface, more modulations and an arpeggiator[3].

The clone developed by Black Corporation was unveiled at the SuperBooth 2018. The name KIJIMI refers to K (Kobol)-J(Japan)-M (Module). It contains 8 analog voices with CEM3340 VCOs and SSM2240 VCFs, plus two LFOs. Offers aftertocush function which is not present in the RSF original[8].



(a) MOS-LAB Kobol Expander III



(b) Black Corporation KIJIMI

Figure 3: Semi-modular systems

2.1.2 Modular Systems: Just VCO.

Both Buchla[9] and Moslab[10] developed recreations of the RSF Kobol VCO: Buchla VCO and Mos-Lab 980. There is very little information about either, beyond a few schematics on the web, although most are unreadable (as in the case of the Buchla VCO), and unfortunately both are discontinued.



(a) Buchla Dual Vco



(b) Mos-Lab 980



(c) La67 WFM

Figure 4: Modular Systems

La67-WFM: The WFM's waveshaper is a modified version of the RSF Kobol's waveform crossfader. The core circuit remains largely unchanged, with only slight adjustments. Consequently, it faithfully reproduces the Kobol's waveshaping sound and its distinct quirks[11]. This one, unlike the Behringer Kobol, uses the CoolAudio V3340 OTA. The potentiometer has 12 positions, not 13.

The Auza WavePackets module also features a continuous waveform oscillator sim-

ilar to the RSF Kobol, but with some different waveforms. The Mutant Instrument Braids also includes an oscillator inspired by this type of waveshaping among its sound banks.

2.1.3 Virtual Instruments

PolyKBIII

The PolyKBIII by XILS-lab is a software synthesizer that accurately emulates the original Polykobol, known for its lush analog tones and complex modulation capabilities. It offers a versatile sound engine with multiple filter types and powerful modulation sources, making it ideal for extensive sound design. The user interface is intuitive, providing easy access to features while maintaining the essence of the original hardware.[7].



Figure 5: PolyKBIII VST3 plugin graphical user interface

2.2 Previous related works

Initially, it was considered to carry out work based on the analysis of the original schematics of the synthesizer. However, this idea has been discarded for now because detailed information about the RSF Kobol Expander are very scarce and incomplete, and it has been almost impossible to find a legible version of the original schematics. The information found online mainly comes from well-known and reputable forums and websites dedicated to electronics and synthesizers and adhering to the DIY philosophy, such as ModWiggler.com and Yusynth.net, where experts in the

field share information. The information is very interesting, useful, and sometimes detailed, but it lacks an official character, and therefore is not 100% reliable from an engineering point of view. Some users, like modular synth maker and engineer Mark Verbos, have taken on the task of recreating their own version of the VCO schematic, which is the one that interests us. However, the component values are an estimation since, according to Mark himself, the originals cannot be read well, and access to the modular synthesizer itself is complicated. Appendix A includes a copy of the original schematics and the version generated by M. Verbos of the VCO.

Unlike Moog synthesizers, for which there is an extensive academic literature, the existing bibliography around the Kobol is practically nonexistent, especially in the academic field. The only academic work published to date can be found in the UPF university repository, and it is published in Spanish. The title is 'Kobología' and it is the Final Degree Project of Gustavo Castelo [12]. Kobología, focuses on the virtualization of the RSF Kobol synthesizer, replicating its unique oscillation characteristics with the goal of preserving the sonic essence of the original hardware in a virtual environment. Castelo's work serves as a key reference making significant contributions to the academic field, particularly in sound synthesis and digital audio modeling. This 'Kobología' thesis work details the prior analysis of the synthesizer and the process of creating a sample bank recorded directly from the actual synthesizer, followed by the processing of these sounds and the eventual development of the digital instrument, with Kontakt being the primary tool used.

Another work related to the RSF Kobol oscillator, has been developed in the context of MTG at UPF by researcher Frederic Font, who has modeled the RSF Kobol oscillator's functionality in JavaScript. The source code is available on GitHub repository [13]. For convenience, the complete code for the three main files that make up the oscillator's code has been added in Appendix B of this thesis. This JS project serves as the starting point or baseline to achieve a virtualized VCO that is as accurate as possible to Behringer's recreation.

A sawtooth wave is used as the base signal, which is a predefined waveform in JavaScript using the `OscillatorNode`. This type of wave has a linear upward slope

followed by an abrupt drop, and it's widely used in sound synthesis. Then, it is processed through a rectifier and comparator using **AudioWorklet** nodes. The processed signals are mixed and their amplitudes adjusted via gain nodes. The final signal is visualized on a canvas using an **AnalyserNode**. This can be seen in Figure 6.

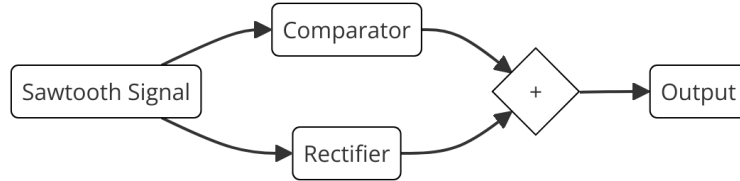


Figure 6: Oscillator structure.

On one hand, the **RectifierProcessor** modifies the input signals based on an adjustable threshold. If an input signal sample is below the threshold, it passes through unchanged. If it is above the threshold, a rectification process is applied. On the other hand, the **ComparatorProcessor** compares signals with a threshold. It transforms the input signal into one where values below the threshold become -1.0, while those above the threshold become 1.0. This makes it an ideal tool for generating a square wave from a continuous audio signal, useful in digital waveform creation.

In the main function, **updateWaveform(value)** controls the dynamic mixing of the two signals (rectified and compared) based on the user-provided input:

- Section A: If the value is low (below 0.3), the rectified signal is mainly used. The comparator's gain ($\text{compGain} = 0$) is disabled, the rectifier's gain ($\text{rectGain} = 1$) is enabled, and the rectifier's threshold (rectThr) is adjusted to vary the waveform.
- Section B: If the value is between 0.3 and 0.6, the comparator and rectifier signals are progressively mixed. The gains of both change gradually, and the rectifier's threshold is set to 1, allowing the comparator to have more influence on the output.
- Section C: If the value is high (above 0.6), the compared signal dominates. The rectifier's gain is reduced to 0, while the comparator's gain is set to 1, and the comparator's threshold is varied to adjust the shape of the output signal.

Figure 7 shows 10 different types of waveforms generated by the oscillator, based on the control parameter `value`, with a range between 0 and 1. It can be observed —although with nuances— that some waveforms are similar to those we aim to obtain. The first image corresponds to a triangular wave, and this one is exact, while the fourth image, the sawtooth wave, is inverted compared to the original. The seventh image shows a square waveform but with a pulse width greater than desired. The rest of the wave images differ quite a bit more from what we aim to recreate and will need to be corrected and adjusted, as will be seen in the next chapter. It can be said that although there is a significant difference between the objective and what is generated, this is a good starting point to develop the virtualization of the oscillator.

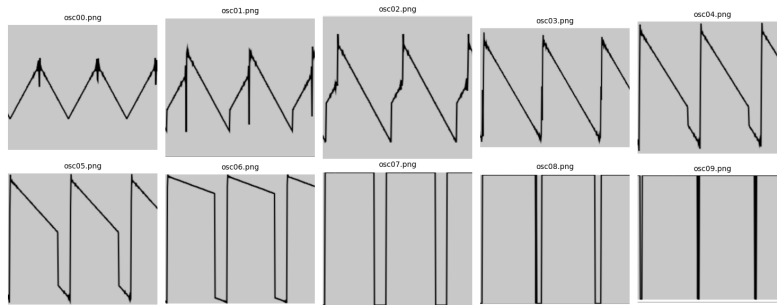


Figure 7: JS oscillator waveforms

Chapter 3

Methodology

3.1 Audio plugin implementation

3.1.1 JUCE framework

JUCE (Jules' Utility Class Extensions) is a widely-used C++ framework designed for creating audio applications and plugins. Developed by Julian Storer, JUCE has become a cornerstone in the audio software industry due to its flexibility and extensive feature set. It supports cross-platform development, enabling the creation of applications that run on Windows, macOS, Linux, iOS, and Android from a single codebase. This makes it a powerful tool for developers looking to build everything from simple audio tools to complex digital audio workstations (DAWs) and plugins.

An audio plugin is a software component used to add functionality to a digital audio workstation (DAW) or other audio software. These plugins are often used to process audio signals in real-time, such as adding effects (reverb, delay, equalization) or simulating virtual instruments. Audio plugins follow specific formats to be compatible with various platforms and software environments [14].

The most commonly used and well-known communication protocols are: VST, VST3 (both are Steinberg SDK standards and are cross-platform), AU (Apple Unit format), AAX (from Avid's Pro Tools software), and LV2 (a popular Linux standard).

Excluding LV2, the other formats are the most commercially used. A common practice among audio developers is to use wrapper libraries, with JUCE being one of the most recognized and classified as an industry standard, as it guarantees support for VST3, AU, AAX, and LV2.

The DAW is the host application that provides access to the computer's audio inputs and outputs. For a synth plugin, the DAW provides it with MIDI events that let the synth know which notes to play. We need a protocol to enable communication between the plugin and the DAW host program, so that the plugin can notify the host of any changes (for example, when the user interacts with the plugin's interface) and allow the DAW to specify what actions to take.

One of the main advantages of JUCE is its cross-platform support, allowing the creation of audio applications that work on Windows, macOS, Linux, iOS, and Android from a single codebase. This reduces the time and effort required for development and maintenance across different platforms. Additionally, JUCE provides a comprehensive set of classes for audio tasks such as input/output, MIDI processing, and audio effects. This feature allows for the efficient integration of complex audio functionalities without needing to build them from scratch, enhancing both development speed and product reliability.

JUCE is particularly effective for developing audio plugins in formats like VST and AU, crucial for professional audio production. Its unified API simplifies plugin creation and ensures compatibility with major digital audio workstations (DAWs). The framework is also optimized for real-time audio processing, which is essential for minimizing latency and ensuring smooth performance during playback and recording.

The GUI toolkit included in JUCE is another strong point, offering tools to design custom and visually appealing interfaces for audio applications. This capability helps in creating professional-grade software that meets high functional and aesthetic standards. Moreover, JUCE benefits from a strong community and extensive documentation, aiding new users in learning quickly and resolving issues efficiently. Its modular nature keeps the codebase lightweight by including only necessary com-

ponents.

Lastly, JUCE offers flexible licensing options, including both open-source and commercial licenses, making it adaptable to various projects and business models. These combined advantages make JUCE a powerful and versatile tool for audio programming, suitable for both beginners and experienced developers.

3.1.2 Plugin architecture

The basic behavior and architecture of the `PluginProcessor` is described in Figure 8. This `PluginProcessor` class is simply an extension of the `juce::AudioProcessor` object. It inherits core audio processing functionality, allowing customization for specific plugin needs, such as handling parameters and managing audio data flow. After constructing the plugin, its basic properties and characteristics are inspected, such as `acceptsMidi`, `getNumPrograms`, `hasEditor`, and others. A call to `prepareToPlay` is made, where the sample rate and audio buffer size are specified, memory is allocated, and the state is reset. Normally, allocating resources is done in the constructor, but the workflow for plugins is a bit different. `ProcessBlock` contains the bulk of the program logic and is the most important part. The most crucial method is `processBlock`. It is called hundreds or even thousands of times per second by the host application and is where your plugin performs all its tasks. When the DAW stops playing audio, a call to `releaseResources` is made, where the used resources will be freed, as shown in Figure 8.

A concept that must be clear when understanding the structure of a synth plugin is MIDI. Simplifying greatly, and applying it to this particular case of a virtual instrument, a synthesizer is a program that transforms a series of MIDI data and information into an audio stream at its output, usually at a rate of 44,100 or 48,000 audio samples per second. A synthesizer not only receives a stream of data at its input but can also receive information about parameter changes, which are related to the synthesizer's user interface, such as knob, slider, and other control information. When these controls are adjusted by the user, they will affect how the synthesizer generates audio.

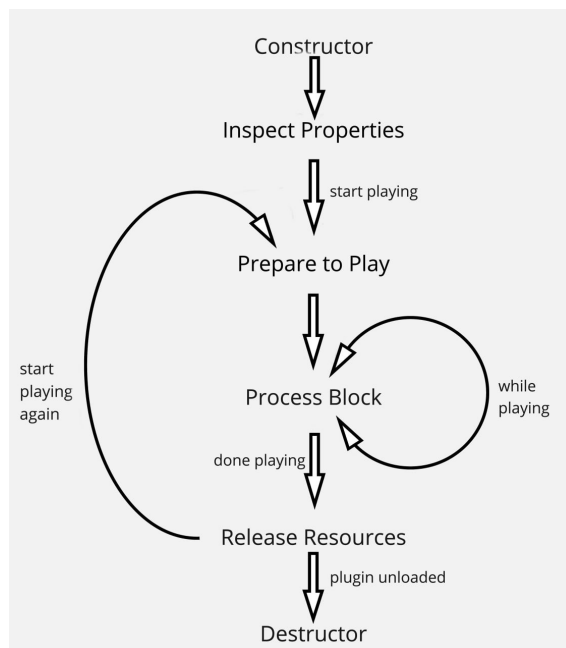


Figure 8: Plugin lifecycle.

MIDI stands for Musical Instrument Digital Interface and was developed in 1980 as a communication protocol between instruments via a basic serial cable. Due to its usefulness, it became a standard in the music industry for connecting electronic musical devices and for handling information in a DAW. Its use continues to this day, and in 2020, MIDI 2.0 was released, introducing significant improvements over the original protocol. In summary, MIDI is not sound per se, but it provides information regarding the sound that should be played.

MIDI information includes which of the 16 possible channels transmits the data. It also specifies the type of command to be executed and on which channel. Commands include, for example, Note On (8x), Note Off (9x), and Control Change (CC) parameters, which transmit information such as Modulation Wheel, Sustain Pedal, and more. Understanding how MIDI works is crucial for developing an instrument like the one discussed in this thesis.

MIDI handling occurs in the most important block, `processBlock`, which will be dealing with `juce::AudioBuffer` and `juce::MidiBuffer` that contains the incoming MIDI messages to be processed. Each time the host application requires new audio from the plugin, it performs an audio callback, calling `processBlock`. This

happens hundreds or even thousands of times per second, depending on the size of the audio blocks being handled. In digital audio, the audio stream is divided into smaller pieces called blocks, as working with all 44,100 samples in a second is inefficient. Thus, the plugin processes one block of samples at a time, with 128 samples being a typical size, although it can be larger or smaller. The smaller the block (also known as buffer size), the more often `processBlock` needs to be called. As always, it is important to balance latency and overhead.

When a new MIDI message is received, it is passed to `processBlock` through the `MidiBuffer` object. MIDI messages are not frequent but occur occasionally, typically only a few times per second, so the `MidiBuffer` is usually empty. The MIDI metadata also includes a timestamp indicating when the message occurs, which is the number of samples relative to the beginning of the `AudioBuffer`. The timing of a MIDI event can occur at any point relative to an audio block. To manage this information and ensure that the event occurs at the correct time and not at the end or start of the audio block, the strategy used is to split the `AudioBuffer` into smaller chunks. We first process any MIDI messages at the start of the buffer, render an audio chunk, then handle the next message, render the next chunk, and so on. This partitioning is handled by the `splitBufferByEvents` method. Within `splitBufferByEvents`, MIDI handling (`handleMIDI`) and audio rendering (`render` method) occur. The `midiMessages` may contain information about more than one MIDI event, and they are ordered chronologically.

For each MIDI event in `midiMessages`, the audio up to the timestamp of that event is first processed using the `render` method. Then, the event is processed using `handleMIDI`, which receives the bytes that make up the MIDI message. This loop continues until all messages are processed. Finally, if there is any remaining audio information in the `AudioBuffer`, the last chunk of audio in the buffer is rendered. This ensures that any audio chunk rendered is always between two MIDI messages, preventing the audio rendering from being interrupted. Lastly, at the end of `splitBufferByEvents`, `midiMessages` is cleared so it can be used again in the next method call. This described behavior is illustrated in Figure 9.

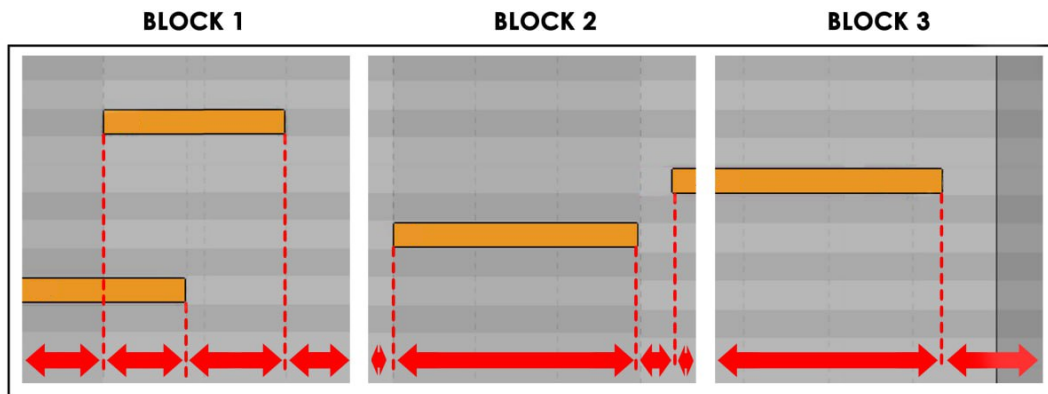


Figure 9: The block is segmented each time a MIDI event is triggered.

3.1.3 Synthesizer architecture

After presenting the general architecture of the plugin, the following section describes the classes that make up the synthesizer. These classes are interrelated and allow for the proper management of the audio flow. This relationship can be observed in Figure 10, where the structure and collaboration between them to carry out the synthesis process are detailed.

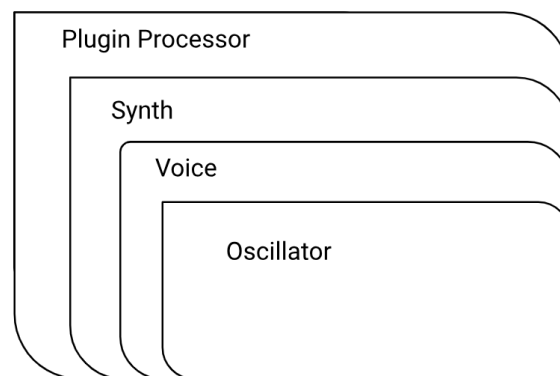


Figure 10: Synth structure

The `Synth` class is the main synthesizer class and is privately instantiated within `PluginProcessor.cpp`. Its function is to manage and coordinate the resources needed for synthesis and audio rendering, including the allocation and deallocation of memory and resources. Through the `render()` function, it delegates the audio

processing to its **Voice** class attribute, specifically calling `voice.render`. It also handles MIDI messages, executing specific actions based on the received information, particularly **NoteOn** and **NoteOff** messages. For example, when a **NoteOn** message is received, the parameters of the Voice class's oscillator (such as amplitude, increment, and note) are prepared to begin audio processing. Additionally, it includes a safety function like **protectYourEars**, which prevents the sound from exceeding safe levels when working with headphones, helping to avoid potential hearing problems.

The **Voice** class manages the playback of a single note at a time. This design allows the creation of a polyphonic synthesizer, where the **Synth** object can have a Voice instance for each note that is playing. This approach is a common practice in synthesizer development. The primary role of the Voice class is to produce the next output sample for a given note. It contains a private oscillator attribute, which generates the audio waveform. The audio rendering is handled through the `render()` function, which calls `osc.nextSample(waveformValue)` to compute the next sample for playback, based on the input parameter that corresponds to the position of the oscillator's waveform selection knob.

3.1.4 Oscillator adjustment

Once the body and logic of the digital synthesizer have been implemented, it's time to add the oscillator and carry out the tuning and correction process of the existing oscillator to make the instrument functional. We start with the previously described Javascript code and adapt it to the language we are using, C++ within JUCE framework. The original functions of the oscillator in JS that we started with, will remain: **updateWaveform**, **Comparator**, **Rectifier**. A new function will be added, which will calculate **nextSample**, and this is where the sawtooth wave is created, and the gain and threshold parameters are updated.

To begin with the tuning process, a tool will be used to always provide a visual reference of the signal we want to virtualize. In this case, the output from VCO1 of the analog synthesizer. The possibility of using a digital oscilloscope was considered, either as a standalone program or as a virtual oscilloscope within a DAW. This idea

was discarded because it is somewhat cumbersome to rely on a computer for this. Due to its convenience, a hardware oscilloscope has been chosen, specifically the Korg NTS-2. A connection is established between the VCO1 output via a patch cable and one of the two inputs of the Korg. The screen isn't very large and doesn't contain the same detailed information as a lab oscilloscope, but it serves perfectly for this purpose, allowing us to visualize the waveforms of the received signals. This will give us a fairly accurate idea of the signal. Also, for convenience and to avoid having to rely on a DAW all the time, since it can consume significant computer resources, we will work with the standalone version of the audio plugin. The virtually generated signal will also be sent to the oscilloscope through the computer's audio output using a stereo minijack cable.

The values related to the waveform selection positions can be divided into three sections, just as in the original implementation of the oscillator. Zone A is where only the rectifier will be activated, and this is the zone where the transitions will occur, from triangle to sawtooth, passing through a shark-like waveform. Zone B spans from the sawtooth reference wave to the square wave, passing through a sawsquare waveform, a double sawtooth, and another type of sawtooth with different amplitude and slope. The last section, Zone C, is where only the comparator will act, and here the square wave will gradually decrease its pulse width until it becomes a very narrow signal, almost like an impulse.

The first signal to be compared and adjusted will be the sawtooth, as it is through this waveform, and via the processes already explained, that the entire continuous range of the oscillator's signals will be formed. Compared to the Kobol, the sawtooth signal is inverted, so the first adjustment will be to correct this by modifying the virtual oscillator to now have a positive slope.

The next reference to adjust will be the triangle wave. For the triangle, the gain of the comparator is set to zero, so it's as if it isn't acting; only the rectifier function will operate. What has been changed is the branch of the rectification function that acts when the processed signal exceeds a certain threshold level, which is calculated and set based on the waveform selection position. In this case, the variable

`currentSplitPosition` was modified, which will affect the threshold for rectification. After these modifications, and by inspecting both in the time domain and visually, we can confirm that the triangle is quite well adjusted, as are most of the intermediate waveforms between the sawtooth and the triangle. Simply by adjusting these parameters, we can also see how the waveforms in Zone B are very close to the desired results.

For Zone C, the value of the variable `currentSplitPosition` was modified, which will affect the comparison threshold. If the input signal is greater than the comparison threshold, the signal will be set to zero; otherwise, it will be set to one. This logic has been modified so that the square and PWM signals are as close to the original as possible since the code originally sets the range of these signals between -1 and 1. This range change will only be applied to Zone C, as Zone B works correctly with the original range. The threshold tuning was entirely manual, with some calculations involved, adjusting and testing values to ensure that the pulse widths are as close as possible to those of the Kobol.

The complete code resulting from the adaptation of the oscillator logic to be included on the synth structure, and the oscillator adjustment is available in Appendix C. Since adding all the plugin's code to the appendix section is excessive, only the 'Oscillator.cpp' file will be included.

3.1.5 Plugin GUI

At first, the user interface or GUI was intended to be designed in the usual way in JUCE, which is through the `PluginEditor` class (along with `PluginProcessor`, these are the core of any project within this framework). This involves designing, programming its behavior, and rendering elements individually, which are then integrated into the `PluginEditor`. Elements such as knobs (both the one emulating the potentiometer that selects the desired waveform and the one controlling the output level of the virtual instrument signal), the integration of a MIDI keyboard, a possible oscilloscope to visualize the waveform of the generated signal, and other aesthetic elements like the background. This task is somewhat tedious, especially the integra-

tion of the keyboard and oscilloscope. One of the objectives of the thesis is not to design a plugin entirely from scratch using JUCE, without using pre-existing tools that facilitate the development of such programs. The goal is rather to develop a virtual instrument, with JUCE being just another tool (which we need to learn to use) for this and future audio programming tasks. Therefore, it has been decided to use a module developed for this purpose. This module is 'foleys_gui_magic', which allows for very convenient editing of the GUI with a more professional finish.

The JUCE module 'foleys_gui_magic' is developed under the company name Fo-leysFinest [15]. Credit should be given to its author, Daniel Walz. This tool provides a series of libraries and JUCE modules for developing graphical interfaces for audio. It follows a WYSIWYG (What You See Is What You Get) style. It allows integration with the main types of audio plugins: VST, AU, and standalone applications. The module is an open-source project and is used both in the industry for commercial plugin development and in renowned institutions such as the CCRMA at Stanford University[16].

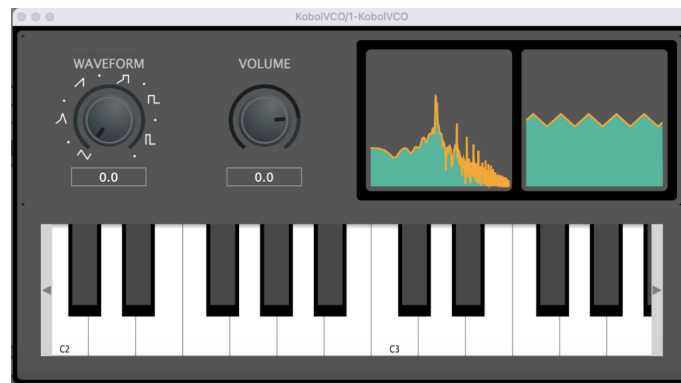


Figure 11: VST3 plugin graphical user interface

The outcome of this process is a fully operational audio plugin that successfully integrates our oscillator emulation within a real-time environment. The plugin has been compiled as both VST and AU formats, ensuring compatibility and integration with a wide range of digital audio workstations (DAWs). Additionally, a standalone version of the application has been developed. The whole project and plugins are also available on GitHub for broader access and collaboration ¹.

¹KobolVCO repository <https://github.com/Hardkoala/KobolVCO>

3.2 Signal capture

In this section, the process followed for sample taking that will be used to compare the original instrument with the recreated one will be described. The next chapter will discuss this in more detail. Samples will be taken from both instruments, and various recordings will be used to conduct an objective and a subjective evaluation, where the similarity between the original analog sound of the Behringer Kobol and the synthetic sound recreated through the virtual instrument will be assessed.

Signal recording

The following devices are involved in the signal recording of the Kobol: the Arturia Keystep 37 controller keyboard, the Behringer Kobol Expander synthesizer, the SSL2 sound card by Solid State Logic, the oscillator provided by the Korg NTS-2 multifunction tool, and a computer. The process is the usual one in these cases. The MIDI output of the Keystep 37 will be connected to the input of the Kobol using a 5-pin MIDI cable. On the other hand, the output from the VCO1 oscillator of the Kobol will go to one of the inputs of the Korg. The Korg has thru connections that replicate the input signal and pass it unaltered through the thru outputs. From this thru output, we will finally send the signal to the sound card, which is connected to the computer. A DAW has been used to digitally and directly record these incoming signals from the synth through the SSL2. During the recording process of the signals from the VCO1 output of the Kobol, the levels of these signals were manually adjusted using the Korg oscilloscope, so that they are not too different in this aspect compared to the signals that will be recorded from the virtual instrument. Nevertheless, after processing the recorded signals, another adjustment will be made by normalizing them.

The recording of the plugin's audio is a simple process and is done directly through the DAW, where the virtual instrument is loaded as a VST plugin. This plugin is controlled similarly to the hardware synthesizer, using the same MIDI keyboard to send notes and control commands. Additionally, a mapping has been set up between the plugin's virtual potentiometer and one of the physical knobs on the MIDI

controller. This allows the user to manipulate the virtual instrument's parameters in real time.

Three reference notes across different scales: A2, A3, and A4, will be sampled. For each of these three notes, and for both instruments, a record of 12 signals will be taken, each corresponding to the 6 main waveforms and the 6 intermediate ones, resulting in a total of 36 samples for each of the two instruments. A processing algorithm will be applied that will allow us to capture 1 and/or periods of the signal. For practical reasons, the last waveform, which would be number 13, will not be considered, as implementing this logic in the VST would be unnecessary since that waveform can be found between waveforms 10 and 12, making it redundant. In fact, for example, the waveform selector of the oscillator in the mentioned La-67 WFM module does not include it.

It should be noted that the notes must match in frequency and therefore in period, and while this is not a problem for the plugin, for the B. Kobol, it will be necessary to check that the frequency potentiometer is correctly adjusted and tuned. This tuning is carried out using the tuning tool provided by the Korg NST-2.

Signal processing

Once the 72 total audio samples for both instruments have been recorded, it will be checked to ensure that all are perfectly recorded and captured, since given the large number of samples and the fact that they are taken manually, there may have been some error or mistake. The signals have a transient period at the beginning, which we will clean up. Using processing algorithms, only one period is selected from each audio signal and saved as an audio signal, as it will be easier to work with the processed signal rather than having to process it each time it is used. It is verified through visual inspection of the signal over time that the period has been correctly captured, as some signals that are a bit more complex, such as those with multiple peaks or maxima or that cross zero several times, may not have had the correct period captured. Due to these signals, it is also a good idea to consider the phase of the signals, meaning that those being compared should have a coincident

or aligned start and end point, because if they are not, the time-domain comparison will not be accurate. To achieve this, it will be established that an aligned period is one whose beginning occurs at the first zero-crossing point with a positive slope and is located just before the global maximum of the signal.

Another factor to consider for the signals to be compared directly is that they must be within the same range of values, which is why the signals need to be normalized. Although the levels were already adjusted with the oscilloscope when recording the signals from the Kobol and the plugin, due to the fact that other factors come into play during the recording process, such as the adjustable input gain of the sound card, and also due to the fact that the analog synthesizer and the VST are recorded in different ways (The first is recorded through the aforementioned sound card, and the second is done directly through the chosen DAW.), the final levels of both types of signals will differ. By normalizing both signals, this problem is resolved.

Chapter 4

Results

4.1 Time Domain Evaluation

For the time-domain evaluation of the signals, a single period will be compared, and it is crucial that the signals from the Kobol and the plugin are in phase, meaning the signal samples must be aligned. Therefore, it is established as a reference that a period will begin after the first zero crossing with a positive slope (increasing signals), as more than one zero crossing can occur within the same period, depending on the signal and its complexity. This will be taken into account because the intermediate waves between the six main waves are a bit more complex. The MSE metric has been chosen for calculating the difference between the signals. The Mean Squared Error (MSE) is a commonly used metric to measure the difference between two signals in the time domain, including the comparison of audio signals. MSE is calculated by taking the difference between the amplitudes of two signals at each point in time, squaring those differences, and then averaging them over the duration of the signal, which helps in assessing the fidelity of audio processing algorithms[17].

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

A sample of the results obtained after performing the calculation of this metric for

all audio pairs of note A3 can be seen in Figure 12. Table 2 shows the result for all audios. The results are quite good, so we can conclude for now that, in terms of the time domain and waveform only, there is a very strong similarity between the original signals and those synthetically recreated. The rest of plotting results for notes A2 and A4 are included in Appendix D.

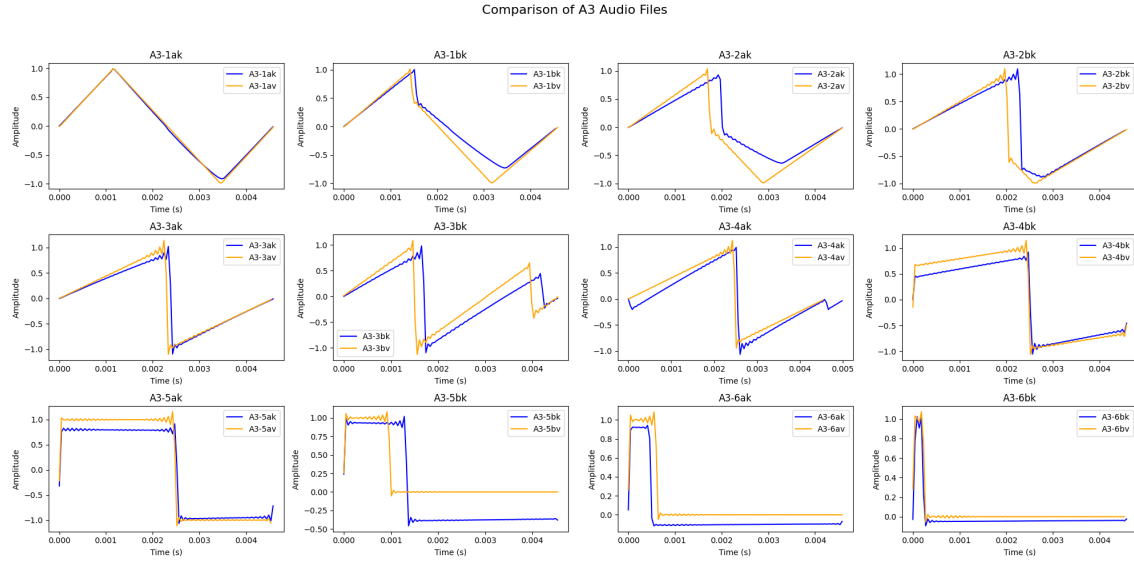


Figure 12: Comparison of 1 period of B.Kobol and plugin for A3 audio signals .

	1a	1b	2a	2b	3a	3b	4a	4b	5a	5b	6a	6b
A2	0.145	0.029	0.091	0.148	0.074	0.149	0.047	0.024	0.057	0.151	0.046	0.011
A3	0.001	0.026	0.089	0.151	0.067	0.189	0.057	0.039	0.048	0.167	0.043	0.009
A4	0.001	0.029	0.077	0.137	0.038	0.206	0.263	0.053	0.057	0.175	0.039	0.012

Table 2: Mean Square Error comparing the signals captured from the B.Kobol and the plugin

4.2 Frequency Domain Evaluation

The frequency domain evaluation will be carried out using a different method from the one previously mentioned. In this case, the signals to be analyzed will be the same sustained note, through which a sweep is performed, passing through the 12 possible waveforms. The note A3 has been chosen as the reference. One recording will be made for the Kobol and another for the audio plugin. The note will be sustained for 24 seconds, and every 2 seconds, a transition to the next waveform position will be made. In the case of the Kobol, this sweep will be done manually

with the potentiometer, taking care since its use is imprecise, and the speed of transitioning from one waveform to another will also be inconsistent. The plugin, run in a DAW, allows for automating these transitions so that they are regular and exact. These two audio files are hosted on Freesound as a pack ¹. Once both 24-second recordings are obtained, a comparison between them will be conducted. Following the evaluation method established in G. Castelo's work [12], an analysis will be implemented using the cosine similarity metric of the MFCC coefficients of the signals.

Cosine similarity measures the angle between two vectors and is used to compare their shape regardless of their size. In audio signals, it helps compare patterns while ignoring differences in amplitude. Additionally, cosine similarity is commonly used to compare MFCC coefficients (Mel-Frequency Cepstral Coefficients), which are features that represent the audio signal's characteristics.

$$\text{Cosine}(x, y) = \frac{x \cdot y}{|x||y|}$$

Figure 13 shows the calculation and representation of the MFCCs, while in Figure 14 both the MSE and the cosine similarity between these coefficients will be represented. It can be observed that the values of this latter metric are not as close to 1 as we would like. In Figure 15, the spectrograms of both sounds are also shown, which will also give us an idea of their components. It is observed that the Kobol has a spectrogram where the harmonics are distributed regularly and proportionally between them, whereas in the other spectrogram this is not the case; there are harmonics where there shouldn't be any, and it will be seen that this is due to Aliasing, among other things.

The evaluation from the frequency perspective is not as good, as can be seen in the graph of the cosine similarity calculation of the MFCC coefficients of the compared audios. The explanation for this is that the frequency spectrum of the waveforms from the Behringer Kobol and the audio plugin will have differences. Several factors

¹Freesound Sweep Pack: <https://freesound.org/people/Valen.malpica/packs/41832/>

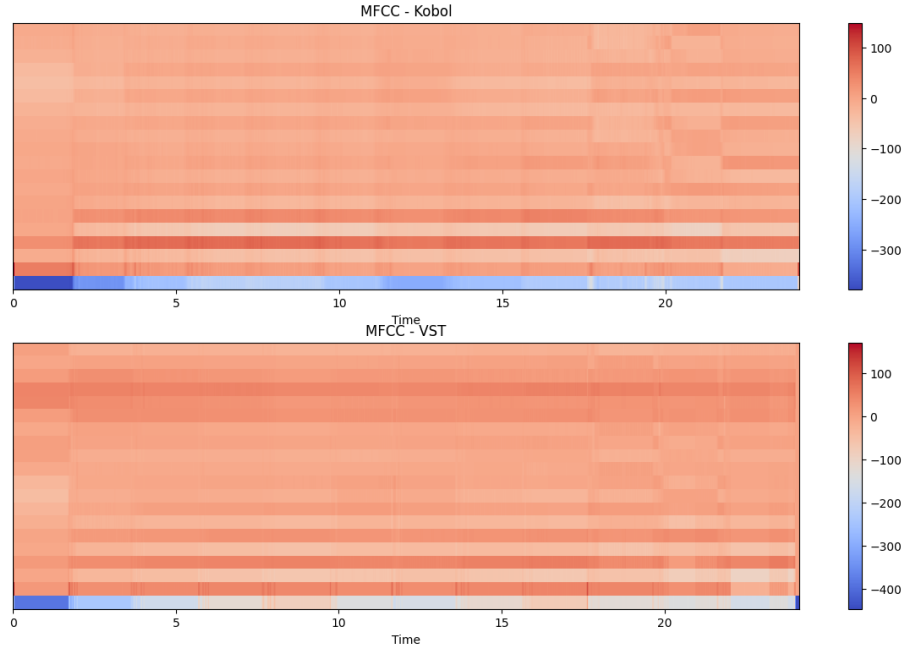


Figure 13: MFCCs of Kobol and plugin signal

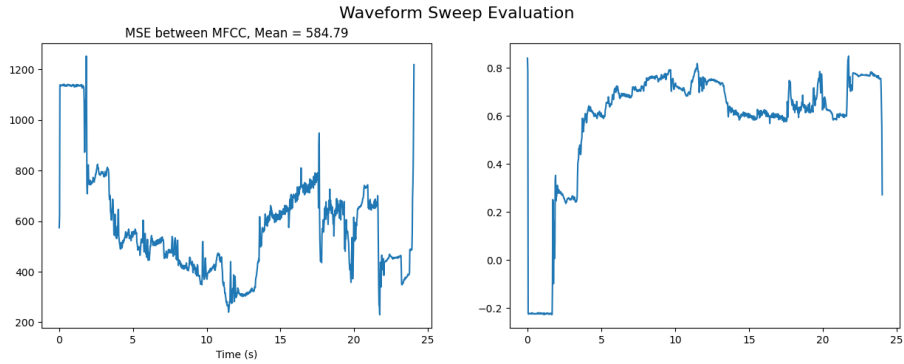


Figure 14: MSE and cosine similarity of MFCC

influence this. The first factor is that the waveform from which the rest are recreated, the sawtooth, is not band-limited, which introduces unwanted frequencies between the harmonics in the recreated signals' spectrum. A possible correction for this is to use a better and more efficient DSP algorithm for the sawtooth that is band-limited. Classic techniques such as BLIT (bandlimited impulse trains) introduced by Stilson and Smith [18], which was state-of-the-art at the time but is no longer used in more recent synthesizers, could be employed. Another option for creating a

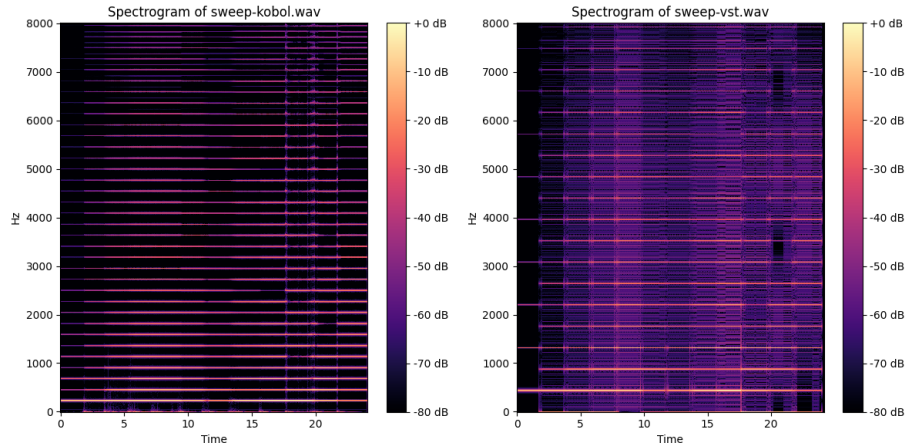


Figure 15: Spectrogram of Kobol and plugin signal

sawtooth is the Differentiated Parabolic Wave technique [19]. Nowadays, the most popular methods are based on BLEP (Band-Limited Step function), which is an enhancement of the BLIT method, and there are several variations like “MinBLEP” or “PolyBLEP”.

Another factor affecting the poor metrics is the recording method used for the signal from the B. Kobol, as the appropriate equipment was not available. The signal should have been recorded using a CV signal to control the waveform more precisely, but since that was not the case, it had to be done manually, resulting in a loss of precision both in the selected waveform and in the transition process from one wave to another, which was irregular, especially as the Kobol is an analog device.

4.3 Subjective evaluation

To carry out the subjective evaluation of the audio plugin, it was decided to use a long audio track, around 36 seconds, one for the original instrument and the other for the virtual VCO synthesizer, replicating the same melody. The strategy is to change waveforms throughout this audio. To establish a reference, the goal is to evaluate the six main waveforms and six secondary or intermediate ones. Since there are 12 signals and a 36-second audio, it doesn’t make perceptual sense to make so many changes in a short time and have each waveform play for so little time. Therefore,

the same audio will be used three times, dividing each audio into four equal parts, which will also correspond to the previously mentioned Zones A, B, and C from the methodology chapter. These sounds can be found in two packs on Freesound, one for the B.Kobol ² and the other for the KobolVCO VST ³.

- For Zone A, the perceived volume in the VST is slightly higher compared to the Behringer. At some point, a separation between notes can be noticed in the VST. The similarity between both is quite high, although there is more harshness in the VST.

- In Zone B, the Behringer Kobol starts with a brighter timbre than the VST instrument. More harshness/saturation is noticeable at the beginning of the VST. The dynamic jump is more pronounced in the second section (9"). The perceived similarity is very close, especially in the third section (17").

- In Zone C, background noise can be heard in both during the first section, being more pronounced in the VST. The perceived sound level is similar in terms of volume. The similarity perception is lower in the last section, where the VST sound feels more compressed (26").

In general, subjectively, it can be said that the level of similarity is high, although there are sections where the Behringer Kobol samples differ slightly from the virtual instrument. There is consistency in the relationship between the different waves and the transition from one to another in the digital instrument.

4.4 Plugin Evaluation

4.4.1 Plugin Validation

A practice that should be done before the final release of a product like this is to test and ensure that the plugin performs well. To verify that the plugin behaves

²Freesound Evaluation Pack 1: <https://freesound.org/people/Valen.malpica/packs/41824/>

³Freesound Evaluation Pack 2: <https://freesound.org/people/Valen.malpica/packs/41826/>

properly, it will be run through a handy tool called Pluginval by Traktion [20]. Pluginval is an open source testing tool used to check the performance and compliance of audio plugins. It provides detailed reports on the plugin's performance, compatibility, and adherence to standards. The configuration involves running the plugin through various tests to ensure it meets the required performance criteria. Results from Pluginval will help identify any issues that need addressing before the final release. The selected options for these tests were: "Validate in process", "Verbose Logging", "Randomise tests", "Stricteness level of 6". These test results can be seen in the Appendix E, that includes the resulting log from the evaluation through Pluginval, and the outcome is that all the tests applied by the program were completed successfully and all passed.

4.4.2 Performance testing: Stress Test

It will give us an idea of CPU consumption in the DAW, and therefore an approximate idea of the resource usage of the virtual instrument. The primary objective of a virtual instrument like this is to be used within a DAW. Therefore, it's essential to consider the plugin's CPU usage, ensuring it does not consume excessive CPU resources. Monitoring CPU consumption gives an indication of the plugin's efficiency. It is important to test CPU usage while the plugin is actively generating sound, as some hosts may pause the plugin if the track is not playing.

A valuable test to perform is the stress test. In a DAW, the plugin is loaded on 10 tracks simultaneously and assign MIDI regions to each track so that all instances of the plugin are rendering audio at the same time and then, observe whether the CPU usage increases linearly (10 times the CPU time) or if it becomes disproportionately higher. One limitation of testing performance in a "how fast can this run?" scenario is that the plugin may benefit from having all its data in the cache. However, when running ten or more instances simultaneously, cache misses may occur, providing a more realistic assessment of performance under typical usage conditions.

A stress test was carried out with varying numbers of tracks loaded in the DAW. A single track with the plugin running only consumes 2% of CPU. When increased

to 10 tracks, it rises slightly to 3%. At 30 tracks, the consumption reaches 4%. It is only when the number of tracks is increased to 60 that we see a more noticeable rise, up to 13% CPU consumption. These results are reflected on Table 3, and it can be concluded from them that the developed plugin is quite efficient.

1 track	10 tracks	30 tracks	60 tracks
2%	3%	4%	13%

Table 3: CPU percentage consumption for different number of tracks

Chapter 5

Conclusions

5.1 Discussion

The main objective of this project has been to develop a functional virtual model replicating the oscillator module of the Behringer Kobol Expander synthesizer, resulting in a more than satisfactory virtualization. The outcome is a functional instrument that can be used as a standalone application or as a digital audio plugin (VST and AU) within a DAW.

Another objective that was worked on and achieved was to carry out a comparison between both instruments through a type of objective and subjective evaluation, both in the time and frequency domains. Metrics such as the calculation of the mean squared error, as well as the cosine similarity of the MFCC between the Kobol and the plugin, were employed. These results have been satisfactory in that the recreated waveforms are very close to the originals in the time domain. However, it was detected that there is a greater difference when comparing the spectral components of both, so the timbre of the instruments will be different and this will be perceptible. Perceptually speaking, it can also be established that while the synthesized sound of the virtual instrument is not identical to the original synthesizer, it can be considered very similar. Lastly, it has been confirmed that the plugin performs correctly through an evaluation test, and with a stress performance test, we see that its operation is

efficient as it consumes few resources.

The necessary knowledge has been acquired and expanded to work in the field of Audio Programming, through JUCE. The basic methodology and logic employed in developing an audio plugin have been worked on and learned. I have been able to understand how to work within JUCE framework, learning to use the resources and tools focused on audio programming that this framework provides.

With this project, a contribution has been made to expanding the existing academic information on the RSF Kobol and its legacy. This will allow future contributions to the community in creating an open-source model as faithful as possible to the original, serving for future academic use.

5.2 Future work

One key direction for future work is to refine the current virtual modeling of the RSF Kobol's oscillator module. This could involve exploring more sophisticated algorithms for simulating the analog behavior of the oscillator to increase accuracy and realism, thereby minimizing discrepancies between the virtual and physical instruments. Other types of modeling could also be explored, such as employing machine learning methods or characterizing the oscillator through neural networks. Another option would be to analyze the original circuitry of the RSF Kobol, though this may be challenging as the currently available schematics are nearly illegible.

The current naive implementation of the sawtooth wave introduces unwanted frequencies in the signal spectrum due to aliasing effect. Using a more efficient DSP algorithm, such as BLEP (Band-Limited Step function), would minimize these issues by producing a cleaner, band-limited waveform.

The current model could be expanded by incorporating additional features commonly found in modern synthesizers. These could include implementing a MIDI learning button, LFO (Low-Frequency Oscillator) for modulation effects, filters such as VCF, and polyphony, allowing the synthesizer to play multiple notes simultaneously. All these improvements would increase its versatility and potential for

complex sound design.

While the current waveform sweep evaluation provides useful insights, it could be further refined. Implementing more objective and accurate metrics to compare waveforms in both the time and frequency domains would give a better understanding of the synthesizer's performance. Additionally, integrating machine learning tools could help automate and improve the accuracy of these comparisons. Also the subjective evaluation could benefit from a more structured approach, possibly involving expert listeners or more formal perceptual tests. Likewise, enhancing the frequency domain analysis with more precise spectral comparisons would provide deeper insights into the sonic differences between the virtual and original synthesizers.

List of Figures

1	RSF Kobol Expander I front pannel	5
2	Behringer Kobol Expander front pannel	7
3	Semi-modular systems	10
4	Modular Systems	10
5	PolyKBIII VST3 plugin graphical user interface	11
6	Oscillator structure.	13
7	JS oscillator waveforms	14
8	Plugin lifecycle.	18
9	The block is segmented each time a MIDI event is triggered.	20
10	Synth structure	20
11	VST3 plugin graphical user interface	24
12	Comparison of 1 period of B.Kobol and plugin for A3 audio signals . .	29
13	MFCCs of Kobol and plugin signal	31
14	MSE and cosine similarity of MFCC	31
15	Spectrogram of Kobol and plugin signal	32
16	RSF Kobol VCO original schematics	43
17	Mark Verbos VCO redrawn version	44
18	Comparison of A2 Audio Files	59
19	Comparison of A3 Audio Files	60
20	Comparison of A2 Audio Files	60
21	MSError between Kobol synth and the audio plugin	61

List of Tables

1	RSF Kobol VCO levels	7
2	Mean Square Error comparing the signals captured from the B.Kobol and the plugin	29
3	CPU percentage consumption for different number of tracks	35

Bibliography

- [1] Terrell, D. L. *Op Amps: Design, Application, and Troubleshooting*, chap. Chapter 4: Oscillators (Newnes, Oxford, 1996). URL <https://www.sciencedirect.com/science/article/pii/B9780750697026500098>.
- [2] Wilson, R. *Make: Analog Synthesizers. A modern approach to old-school sound synthesis* (Maker Media, 2013).
- [3] RSF Synth history (2024). URL http://rsf-synth-official.com/pages/history/history_en.html.
- [4] Howell, S. Rsf kobol expander. <https://www.muzines.co.uk/articles/rsf-kobol-expander/5929?theme=2> (March, 1983).
- [5] Behringer Kobol Expander. URL <https://es.everand.com/article/704968796/Test-Behringer-Kobol-Expander>.
- [6] Behringer Kobol Expander (2024). URL <https://amsynths.co.uk/2024/01/19/behringer-kobol-expander/>.
- [7] Xils-lab PolykbIII VST. URL <https://www.matrixsynth.com/2023/07/mos-lab-kobol-expander-iii-rsf-sn-3561.htm>.
- [8] Bloderer, T. Kijimi RSF Polykobol II Clone. <https://greatsynthesizers.com/en/general/kijimi-rsf-polykobol-ii-clone-made-in-japan/> (Abril,2018).
- [9] RSF Kobol Dual VCO Module. URL https://modularsynthesis.com/buchla/kobol_vco/kobol_vco.htm.

- [10] MOS-LAB 980 Kobol Oscillator . URL <https://modulargrid.net/d/mos-lab-980-kobol-oscillator>.
- [11] La-67 WFM Oscillator User guide. URL https://www.lasesentaysiete.com/_files/ugd/ab6c23_7308001b144441a7b871709b70928b3d.pdf.
- [12] Castelo, G. Kobologia. <https://github.com/gcgus/Kobologia> (commit c553008).
- [13] Font, F. Kobol Oscillator JS code: kobol_osc_js. <https://github.com/ffont/kobol-oscillator> (commit a1b2c3d).
- [14] Vincent Goudard, R. M. Real-time audio plugin architectures (2003).
- [15] FoleysFinest Plugin GUI Magic. URL <https://foleysfinest.com/developer/pluginguimagic/>.
- [16] CCRMA Plugin GUI Magic lecture. URL <https://ccrma.stanford.edu/courses/320c/lectures/2/PGM.html>.
- [17] Z. Wang, A. C. B. Mean squared error: Love it or leave it? a new look at signal fidelity measures. In *IEEE Signal Processing Magazine*, 332–335 (IEEE, January 2009). URL <https://ieeexplore.ieee.org/document/4775883>.
- [18] Stilson, T. S. & Smith, J. O. Alias-free digital synthesis of classic analog waveforms. In *International Conference on Mathematics and Computing* (1996). URL <https://api.semanticscholar.org/CorpusID:2864693>.
- [19] Välimäki, V. & Huovilainen, A. Oscillator and filter algorithms for virtual analog synthesis. *Computer Music Journal* **30**, 19–31 (2006).
- [20] Traktion. pluginval. <https://github.com/Tracktion/pluginval> (commit b107dd2).

Appendix A

Schematics

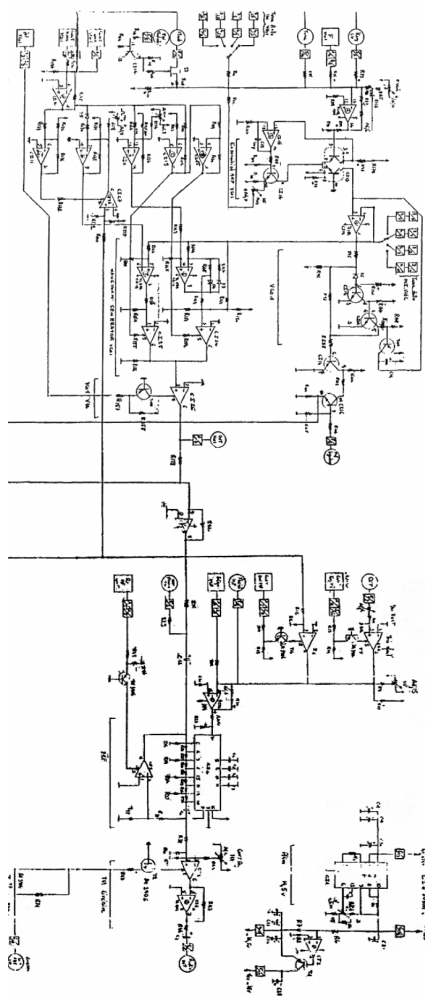
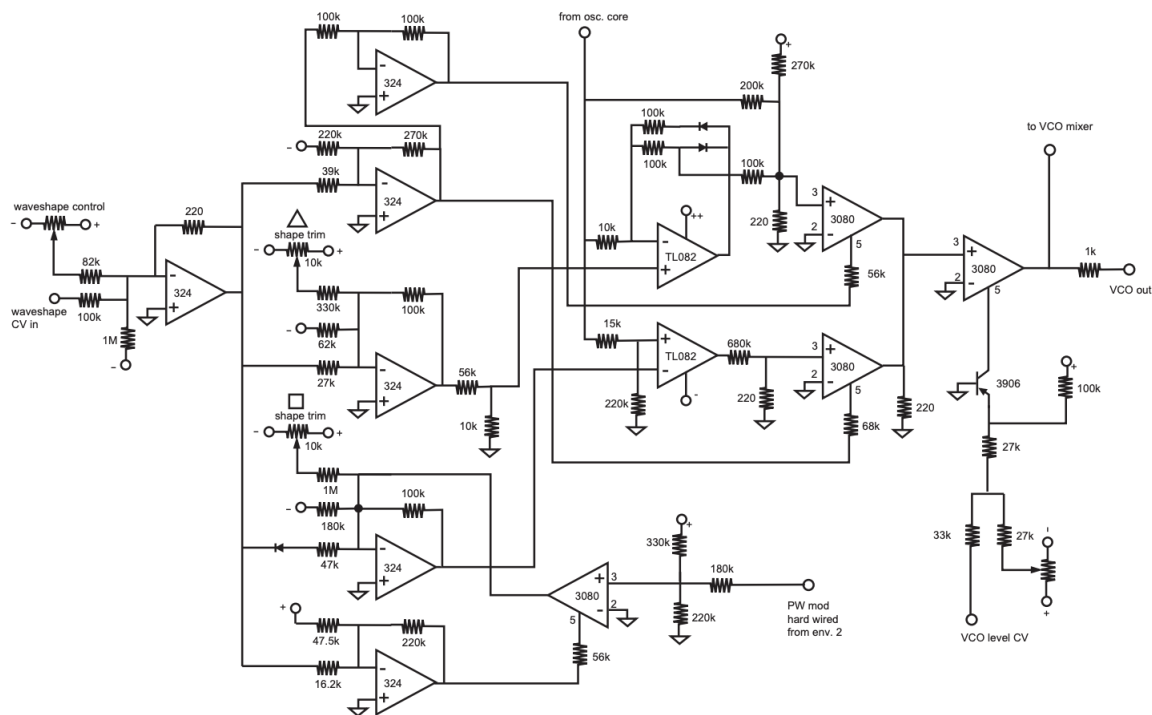


Figure 16: RSF Kobol VCO original schematics

RSF KOBOL EXPANDER VCO WAVE SHAPER - redrawn by mark verbos verbos@simple-answer.com October 19, 2000



NOTES: all OP-AMPS powered by +/- 14.5 volts except TL082 powered by +18(+)/-14.5
wvshape sweeps continuously from triangle to saw to square to pulse
you may want to eliminate the level VCA and replace the 3080 with a regular OP-AMP
I take no responsibility for parts being the wrong values as the original schematic was illegible
all pots were unlabeled but I think 100k linear will work

Figure 17: Mark Verbos VCO redrawn version

Appendix B

Kobol_osc_js Code

This appendix includes the three main files belonging to the code of the Kobol Oscillator implemented in JavaScript by F. Font [13].

B.1 Comparator.js

```
class ComparatorProcessor extends AudioWorkletProcessor {

  static get parameterDescriptors() {
    return [
      {
        name: "threshold",
        defaultValue: 1.0,
        minValue: 0.0,
        maxValue: 1.0,
      },
    ];
  }

  process(inputs, outputs, parameters) {
    const input = inputs[0];
    const output = outputs[0];
```

```
const threshold = parameters.threshold * (0.9-0.5) + 0.5; //
↪ threshold input goes from 0 to 1, but we want 0.5 to 0.9

for (let channel = 0; channel < output.length; ++channel) {
  const inputChannel = input[channel];
  const outputChannel = output[channel];

  for (let i = 0; i < outputChannel.length; ++i) {
    if (inputChannel[i] <= threshold){
      outputChannel[i] = -1.0;
    } else {
      outputChannel[i] = 1.0;
    }
  }
}

return true;
}

registerProcessor('comparator', ComparatorProcessor);
```

B.2 Rectifier.js

```
class RectifierProcessor extends AudioWorkletProcessor {

  static get parameterDescriptors() {
    return [
      {
        name: "threshold",
        defaultValue: 0.5,
        minValue: 0.0,
        maxValue: 1.0,
```

```
        },
    ];
}

process(inputs, outputs, parameters) {

    const input = inputs[0];
    const output = outputs[0];
    const threshold = parameters.threshold;

    for (let channel = 0; channel < output.length; ++channel) {
        const inputChannel = input[channel];
        const outputChannel = output[channel];

        for (let i = 0; i < outputChannel.length; ++i) {
            if (inputChannel[i] <= threshold){
                outputChannel[i] = inputChannel[i];
            } else {
                outputChannel[i] = threshold - inputChannel[i];
            }
            outputChannel[i] = outputChannel[i] + (1 - threshold)/2
        }
    }
    return true;
}

registerProcessor('rectifier', RectifierProcessor);
```

B.2.1 Main.js

```
const audioContext = new AudioContext();
const WIDTH = 800;
const HEIGHT = 200;
const canvas = document.getElementById("oscilloscope");
const canvasCtx = canvas.getContext("2d");
const oscillator = new OscillatorNode(audioContext);
let rectifier = undefined;
let comparator = undefined;
const analyser = new AnalyserNode(audioContext);
const bufferLength = analyser.frequencyBinCount;
const dataArray = new Uint8Array(bufferLength);
const gainRect = new GainNode(audioContext);
const gainComp = new GainNode(audioContext);
const gainMain = new GainNode(audioContext);
canvas.width = WIDTH;
canvas.height = HEIGHT;

const startAudio = async (context) => {
  await context.audioWorklet.addModule('rectifier.js');
  await context.audioWorklet.addModule('comparator.js');
  rectifier = new AudioWorkletNode(context, 'rectifier');
  comparator = new AudioWorkletNode(context, 'comparator');
  oscillator.type = "sawtooth"
  oscillator.frequency.value =
    ↪ document.getElementById("oscPitch").value
  oscillator.connect(rectifier)
  oscillator.connect(comparator)
  rectifier.connect(gainRect)
  comparator.connect(gainComp)
```

```
gainRect.connect(gainMain)
gainComp.connect(gainMain)
gainMain.connect(analyser)
analyser.connect(context.destination);
updateWaveform(0.0);
document.getElementById("waveform").value = 0.0;
oscillator.start();
canvasCtx.clearRect(0, 0, WIDTH, HEIGHT);
draw();
};

window.addEventListener('load', async () => {
  const buttonEl = document.getElementById('button-start');
  buttonEl.disabled = false;
  buttonEl.addEventListener('click', async () => {
    await startAudio(audioContext);
    audioContext.resume();
    buttonEl.disabled = true;
    buttonEl.textContent = 'Playing...';
  }, false);
});

function draw() {
  const drawVisual = requestAnimationFrame(draw);
  analyser.getByteTimeDomainData(dataArray);

  canvasCtx.fillStyle = "rgb(200, 200, 200)";
  canvasCtx.fillRect(0, 0, WIDTH, HEIGHT);
  canvasCtx.lineWidth = 2;
  canvasCtx.strokeStyle = "rgb(0, 0, 0)";
  canvasCtx.beginPath();
```

```
const sliceWidth = WIDTH / bufferLength;
let x = 0;
for (let i = 0; i < bufferLength; i++) {
  const v = dataArray[i] / 128.0;
  const y = v * (HEIGHT / 2);
  if (i === 0) {
    canvasCtx.moveTo(x, y);
  } else {
    canvasCtx.lineTo(x, y);
  }
  x += sliceWidth;
}
canvasCtx.lineTo(WIDTH, HEIGHT / 2);
canvasCtx.stroke();

}

document.getElementById("oscPitch").oninput = (evt) => {
  oscillator.frequency.value = evt.target.value;
}

document.getElementById("rectThreshold").oninput = (evt) => {
  const param = rectifier.parameters.get("threshold");
  param.setValueAtTime(evt.target.value, audioContext.currentTime);
}

document.getElementById("compThreshold").oninput = (evt) => {
  const param = comparator.parameters.get("threshold");
  param.setValueAtTime(evt.target.value, audioContext.currentTime);
}
```

```
}

document.getElementById("compGain").oninput = (evt) => {
  const param = gainComp.gain;
  param.setValueAtTime(evt.target.value, audioContext.currentTime);
}

document.getElementById("rectGain").oninput = (evt) => {
  const param = gainRect.gain;
  param.setValueAtTime(evt.target.value, audioContext.currentTime);
}

document.getElementById("mainGain").oninput = (evt) => {
  const param = gainMain.gain;
  param.setValueAtTime(evt.target.value, audioContext.currentTime);
}

const updateWaveform = value => {
  const rectThr = rectifier.parameters.get("threshold");
  const compThr = comparator.parameters.get("threshold");
  const compGain = gainComp.gain;
  const rectGain = gainRect.gain;
  const splitPointA = 0.3;
  const splitPointB = 0.6;
  if (value < splitPointA) {
    compGain.setValueAtTime(0.0, audioContext.currentTime);
    rectGain.setValueAtTime(1.0, audioContext.currentTime);
    compThr.setValueAtTime(0.0, audioContext.currentTime); // Should
    ↪ this be 0.5??
    const currentSplitPosition = value / splitPointA;
```

```

    rectThr.setValueAtTime(currentSplitPosition,
        ↪ audioContext.currentTime);

} else if (value < splitPointB) {
    compThr.setValueAtTime(0.0, audioContext.currentTime); // Should
    ↪ this be 0.5??
    const currentSplitPosition = (value - splitPointA) / (splitPointB
        ↪ - splitPointA);
    compGain.setValueAtTime(currentSplitPosition,
        ↪ audioContext.currentTime);
    rectGain.setValueAtTime((1.0 - currentSplitPosition),
        ↪ audioContext.currentTime);
    rectThr.setValueAtTime(1, audioContext.currentTime);

} else {
    compGain.setValueAtTime(1.0, audioContext.currentTime);
    rectGain.setValueAtTime(0.0, audioContext.currentTime);
    const currentSplitPosition = (value - splitPointB) / (1.0 -
        ↪ splitPointB);
    compThr.setValueAtTime(currentSplitPosition,
        ↪ audioContext.currentTime);
    rectThr.setValueAtTime(1, audioContext.currentTime);
}

setTimeout( () => {
    document.getElementById("rectGain").value = rectGain.value;
    document.getElementById("compGain").value = compGain.value;
    document.getElementById("compThreshold").value = compThr.value;
    document.getElementById("rectThreshold").value = rectThr.value;
}, 50)

```

```
}
```

```
document.getElementById("waveform").oninput = (evt) => {  
  updateWaveform(evt.target.value);  
}
```

Appendix C

Audio Plugin Code

C.1 Oscillator.cpp

```
/* =====  
Oscillator.cpp  
=====*/  
/*  
  
#include "Oscillator.h"  
#include <iostream>  
  
Oscillator::Oscillator() {  
    phase = 0.0f;  
    rectThreshold = 0.5f;  
    rectGain = 1.0f;  
    compThreshold = 0.0f;  
    compGain = 0.0f;  
}
```

```

void Oscillator::reset() {
    rectThreshold = 0.5f;
    rectGain = 1.0f;
    compThreshold = 0.0f;
    compGain = 0.0f;
}

/* =====
Set Waveform Parameters
=====*/
void Oscillator::setWaveformParameters(float value) {

    if (value < splitPointA) { //0-0.4: rectifier

        float currentSplitPosition= value / splitPointA;
        rectThreshold = currentSplitPosition;
        rectGain = 1.0f;
        compGain = 0.0f;
        compThreshold = 0.0f;

    } else if (value <= splitPointB) { //0.4-0.8:
↪ rectifier+comparator
        float currentSplitPosition = (value - splitPointA) /
↪ (splitPointB -
splitPointA);
        rectThreshold = 1.0f;
        rectGain = 1.0f - currentSplitPosition;
        compThreshold = 0.0f;
        compGain = currentSplitPosition;

    } else { //0.8-1.1: comparator

```

```

        float currentSplitPosition = (splitPointB-
        ↪ value-0.5-(0.6*(value-splitPointB))) / (1.2f -
        ↪ splitPointB);
        rectThreshold = 1.0f;
        rectGain = 0.0f;
        compThreshold = currentSplitPosition;
        compGain = 1.0f;
    }
}

/* =====
Next Sample
=====*/
float Oscillator::nextSample(float position) {

    setWaveformParameters(position); //Potenciometer value
    phase += inc; //Sawtooth implementation

    if (phase >= 1.0f) {
        phase -= 1.0f;
    }

    float sawtoothSginal = (2.0f * phase - 1.0f); // Sawtooth signal
    //Rectifier
    float rectSignal = rectifier(sawtoothSginal, rectThreshold); //
    ↪ Apply Rectifier
    rectSignal *= rectGain;
    //Comparator
    float compSignal = comparator(sawtoothSginal, compThreshold,
    ↪ position); // Apply Comparator
    compSignal *= compGain;

```

```

        return amplitude* (rectSignal + compSignal)/2;
    }

    /* =====
    Rectifier
    =====*/
float Oscillator::rectifier(float inputSample, float threshold) {

    float outputSample;

    if (inputSample <= threshold) {
        outputSample = inputSample;
    } else {
        outputSample = threshold - (threshold+1)* inputSample;
    }

    return outputSample + (1.0f - threshold) / 2.0f;
}

    /* =====
    Comparator
    =====*/
float Oscillator::comparator(float inputSample, float threshold,
↪ float position) {

    threshold = (threshold + 0.2f) * 0.4f;
    float outputSample;

    if (position < splitPointB){
        if (inputSample > threshold){
            outputSample = -1.0f;
        }else{

```

```
        outputSample= 1.0f;
    }
} else {
    if (inputSample > threshold){
        outputSample = 0;
    }else{
        outputSample= 1;
    }
}

return outputSample;}
```

Appendix D

Evaluation results

D.1 Kobol and KobolVCO Waveforms

D.1.1 A2 Waveforms

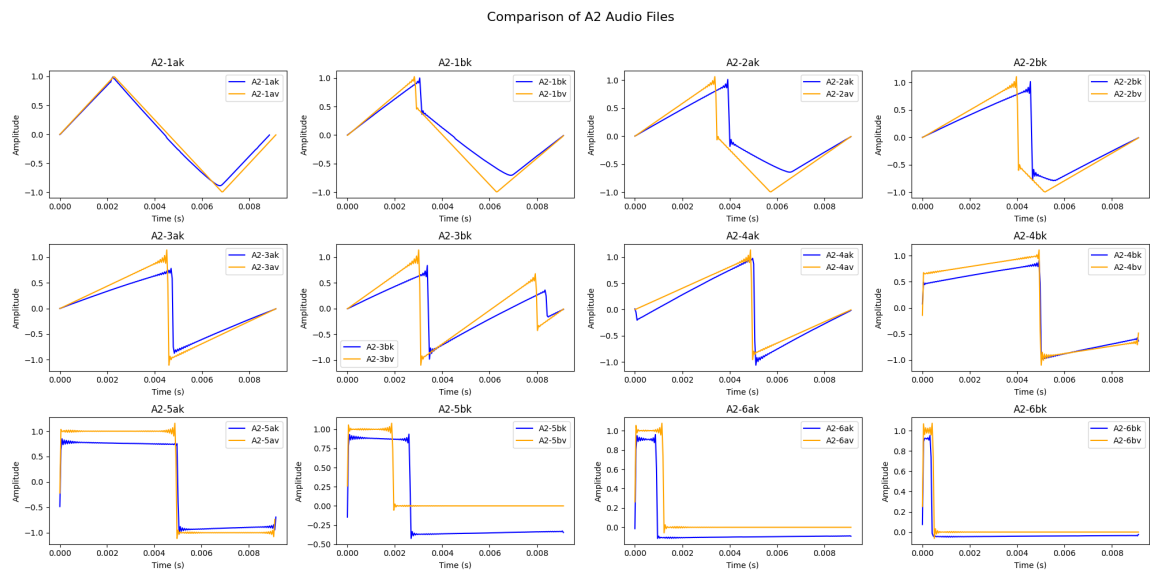


Figure 18: Comparison of A2 Audio Files

D.1.2 A3 Waveforms

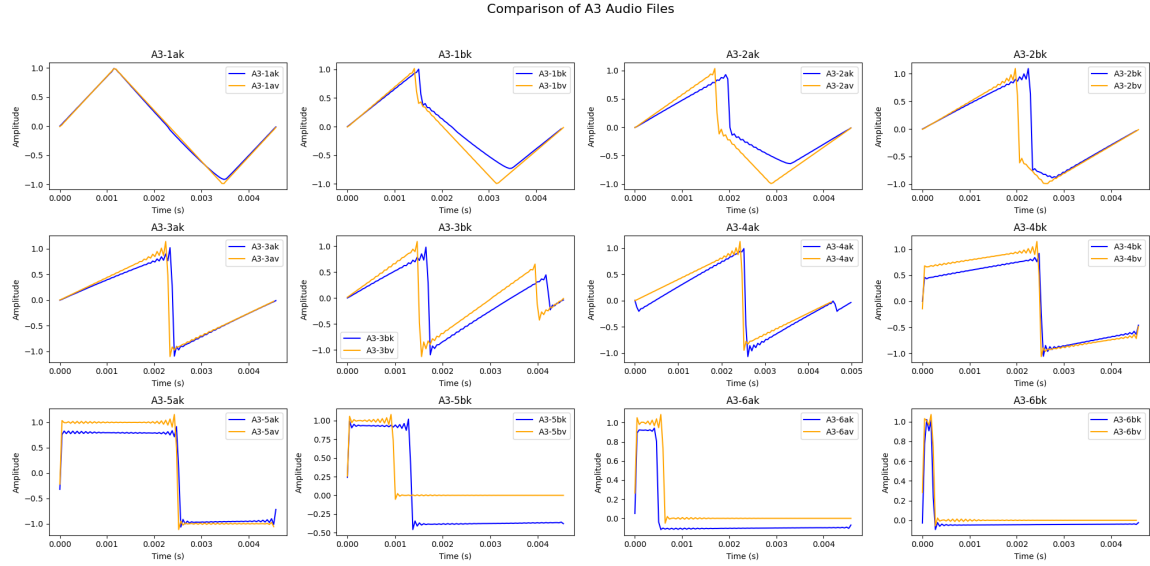


Figure 19: Comparison of A3 Audio Files

D.1.3 A4 Waveforms

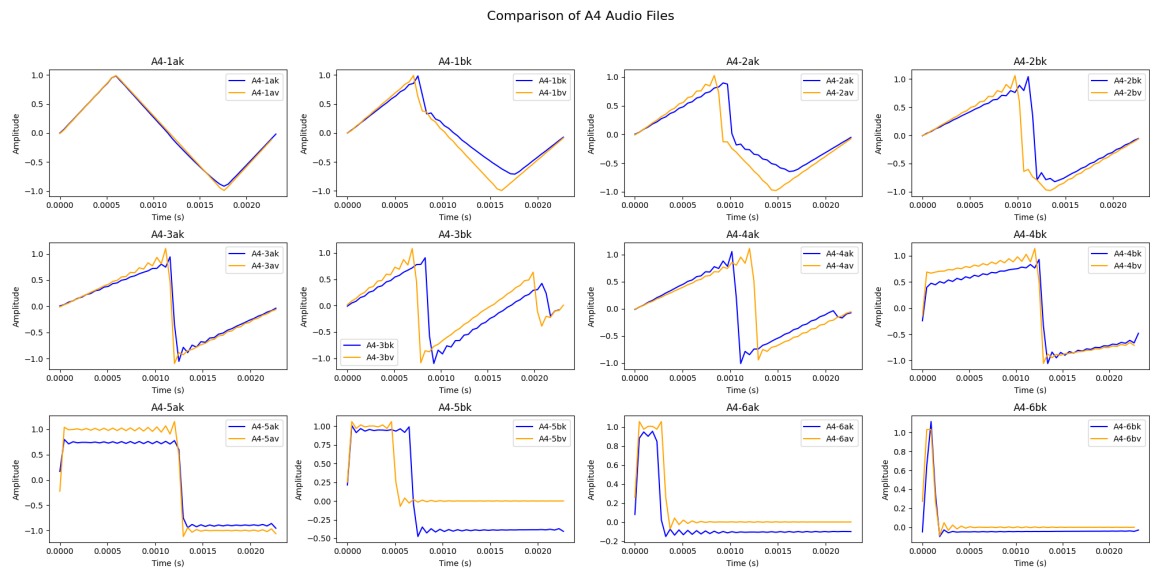


Figure 20: Comparison of A2 Audio Files

D.2 B.Kobol and KobolVCO MSError

	1a	1b	2a	2b	3a	3b	4a	4b	5a	5b	6a	6b
A2	0.1456	0.0297	0.0917	0.1485	0.0743	0.1493	0.0475	0.0247	0.0570	0.1511	0.0464	0.0119
A3	0.0006	0.0263	0.0898	0.1516	0.0676	0.1893	0.0571	0.0391	0.0487	0.1670	0.0436	0.0092
A4	0.0007	0.0290	0.0779	0.1371	0.0388	0.2068	0.2635	0.0533	0.0576	0.1751	0.0396	0.0127

Figure 21: MSError between Kobol synth and the audio plugin

Appendix E

Plugin Evaluation Log

E.1 Plugin Test Evaluation

pluginval v0.2.7 - JUCE v5.4.7

Started validating: VST3-KobolVCO-a10b97f4-4f976ab1

Random seed: 0x2fb

Strictness level: 6

Testing plugin: VST3-KobolVCO-a10b97f4-4f976ab1

valen: KobolVCO v1.0.0

Starting test: pluginval / Open plugin (cold)...

Time taken to open plugin (cold): 193 ms

All tests completed successfully

Starting test: pluginval / Open plugin (warm)...

Time taken to open plugin (warm): 19 ms

Running tests 1 times

All tests completed successfully

Starting test: pluginval / Automatable Parameters...

Testing parameter: 0 - Output Level

Parameter info: index - 0, paramName - Output Level, defaultValue -

↪ 0.8, label - dB, numSteps - 2147483647, isDiscrete - 0, isBoolean -

↪ - 0, isOrientationInverted - 0, isAutomatable - 1,

↪ isMetaParameter - 0, category - 0, all value strings -

Testing accessers

Testing parameter: 1 - Wave Form

Parameter info: index - 1, paramName - Wave Form, defaultValue - 0,

↪ label - VCO, numSteps - 2147483647, isDiscrete - 0, isBoolean -

↪ 0, isOrientationInverted - 0, isAutomatable - 1, isMetaParameter

↪ - 0, category - 0, all value strings -

Testing accessers

Time taken to run test: 0

All tests completed successfully

Starting test: pluginval / Plugin programs...

Num programs: 0

All program names checked

Time taken to run test: 0

All tests completed successfully

Starting test: pluginval / Automation...

Testing with sample rate [44100] and block size [64] and sub-block
↪ size [32]

Testing with sample rate [44100] and block size [128] and sub-block
↪ size [32]

Testing with sample rate [44100] and block size [256] and sub-block
↪ size [32]

Testing with sample rate [44100] and block size [512] and sub-block
↪ size [32]

Testing with sample rate [44100] and block size [1024] and sub-block
↪ size [32]

Testing with sample rate [48000] and block size [64] and sub-block
↪ size [32]

Testing with sample rate [48000] and block size [128] and sub-block
↪ size [32]

Testing with sample rate [48000] and block size [256] and sub-block
↪ size [32]

Testing with sample rate [48000] and block size [512] and sub-block
↪ size [32]

Testing with sample rate [48000] and block size [1024] and sub-block
↪ size [32]

Testing with sample rate [96000] and block size [64] and sub-block
↪ size [32]

Testing with sample rate [96000] and block size [128] and sub-block
↪ size [32]

Testing with sample rate [96000] and block size [256] and sub-block
↪ size [32]

Testing with sample rate [96000] and block size [512] and sub-block
↪ size [32]

Testing with sample rate [96000] and block size [1024] and sub-block
↪ size [32]

Time taken to run test: 11 ms

All tests completed successfully

Starting test: pluginval / Plugin info...

Plugin name: KobolVCO

Alternative names: KobolVCO

SupportsDoublePrecision: no

Reported latency: 0

Reported taillength: 0

Time taken to run test: 0

All tests completed successfully

Starting test: pluginval / Audio processing...

Testing with sample rate [44100] and block size [64]

Testing with sample rate [44100] and block size [128]

Testing with sample rate [44100] and block size [256]

Testing with sample rate [44100] and block size [512]

Testing with sample rate [44100] and block size [1024]

Testing with sample rate [48000] and block size [64]

Testing with sample rate [48000] and block size [128]

Testing with sample rate [48000] and block size [256]

Testing with sample rate [48000] and block size [512]

Testing with sample rate [48000] and block size [1024]

Testing with sample rate [96000] and block size [64]

Testing with sample rate [96000] and block size [128]

Testing with sample rate [96000] and block size [256]

Testing with sample rate [96000] and block size [512]

Testing with sample rate [96000] and block size [1024]

Time taken to run test: 6 ms

All tests completed successfully

Starting test: pluginval / Fuzz parameters...

Fuzz testing parameter: 0 - Output Level

Fuzz testing parameter: 1 - Wave Form

Time taken to run test: 1 ms

All tests completed successfully

Starting test: pluginval / Editor stress...

Testing opening Editor with released processing

Testing opening Editor with 0 sample rate and block size

Time taken to run test: 560 ms

All tests completed successfully

Starting test: pluginval / Basic bus...

All tests completed successfully

Starting test: pluginval / Listing available buses...

Inputs:

Named layouts: None

Discrete layouts: None

Outputs:

Named layouts: Mono, Stereo

Discrete layouts: Discrete #1

Main bus num input channels: 0

Main bus num output channels: 2

All tests completed successfully

Starting test: pluginval / Enabling all buses...

All tests completed successfully

Starting test: pluginval / Disabling non-main busses...

All tests completed successfully

Starting test: pluginval / Restoring default layout...

Main bus num input channels: 0

Main bus num output channels: 2

Time taken to run test: 11 ms

All tests completed successfully

Starting test: pluginval / Open editor whilst processing...

Time taken to run test: 319 ms

All tests completed successfully

Starting test: pluginval / Plugin state restoration...

Time taken to run test: 1 ms

All tests completed successfully

Starting test: pluginval / Plugin state...

Time taken to run test: 0

All tests completed successfully

Starting test: pluginval / Editor...

Time taken to open editor (cold): 248 ms

Time taken to open editor (warm): 251 ms

Time taken to run test: 540 ms

All tests completed successfully

Starting test: pluginval / Editor Automation...

Time taken to run test: 22 secs

Time taken to run all tests: 24 secs

All tests completed successfully

Finished validating: VST3-KobolVC0-a10b97f4-4f976ab1

ALL TESTS PASSED